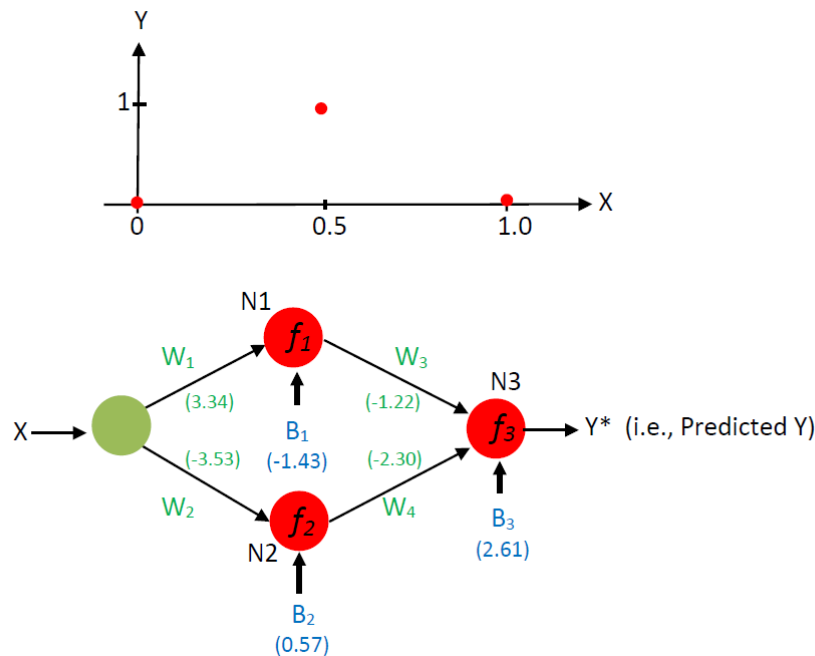


CE4172 Tiny Machine Machine Learning

Tutorial 5 – Backpropagation

In this tutorial, we will discuss how the Weights and Biases in a neural network can be found by using the techniques of gradient descent and backpropagation.

The following figures show the 3-point dataset and a simple artificial neural network that is used to train the model as discussed in tutorial 1.



The following are the hyperparameters used in the artificial neural network.

N1: Weight W_1 ; Bias B_1 ; Activation function $f_1(x) = \ln(1 + e^x)$

N2: Weight W_2 ; Bias B_1 ; Activation function $f_2(x) = \ln(1 + e^x)$

N3: Weights W_3 and W_4 ; Bias B_3 ; Activation function $f_3(x) = x$ (i.e. $Y^* = x$)

Exercise:

Using the Gradient Descent technique (as discussed in Tutorial 3), and the Backpropagation process (describe further below), find the most appropriate values for the various Weights and Biases to be used in the neural network for the 3-point dataset shown above.

Compare your answers with the various Weights and Biases values that are indicated in brackets () in the above model.

CE4172 Tiny Machine Machine Learning

Tutorial 5 – Backpropagation

Gradient Descent technique is used to find the minimum value of a function based on the calculation of its gradient, i.e. to find the point where the gradient is equal (or very close) to 0. As discussed in tutorial 3, we can use it to find the minimum value of a cost function (e.g. , SSR, Sum of square residue), which in turn tell us that the predicted value(s) is(are) closest possible to the observed value(s).

The algorithm of the Gradient Descent can be summarized as follows:

- a. Find the expression that relates the gradient of the cost function (e.g. SSR) against the hyperparameter: $dSSR/dx$ (where x can be the Weight or the Bias)
- b. Estimate an initial value for the parameter: $x(x_0)$
- c. Substitute x into the $dSSR/dx$ expression
- d. Multiply the answer of c with an appropriate Learning Rate (LR) to obtain the step size value:
- e. $step_size = dSSR/dx * LR$
- f. If the value of the step is not near zero, minus the $step_size$ from x used in step c to obtain the new x value:
- g. $new_x = x - step_size$
- h. and repeat the computation from step c using the new_x .

Backpropagation is to extend the gradient descent computation to all the hyperparameters that determine the cost function of the neural network at the output. In the neural network given above, it means that we will need to first find the expressions of the following gradients (which are equivalent to step a) above):

- i. $dSSR/dW1$
- ii. $dSSR/dB1$
- iii. $dSSR/dW2$
- iv. $dSSR/dB2$
- v. $dSSR/dW3$
- vi. $dSSR/dW4$
- vii. $dSSR/dB3$

We then apply the iterative gradient descent calculations to each of these parameters to obtain their new values, and repeat step c) to step e) (of the gradient descent described further above) until the predicted value no longer improve very much, or we meet some other criteria (e.g. maximum number of iteration).

Note: $d[\ln(1+ ex)] /dx = ex/(1+ ex)$